

Math 418
Synthetic Handwriting with Generative Adversarial Networks

Micah Sherry
nbzx@iup.edu

May 7, 2026

Abstract

This project investigates the use of conditional Generative Adversarial Networks (cGANs) for generating synthetic handwritten characters using the EMNIST Balanced dataset. Two architectures were explored: a baseline GAN composed of fully connected layers and a convolutional GAN designed to better capture spatial features in image data. The results show that the non-convolutional model fails to produce meaningful outputs, while the convolutional model successfully generates realistic and diverse handwritten characters across 47 classes.

To quantitatively evaluate generator performance, a separate classification model was trained on real EMNIST data and used to assess the quality of synthetic samples. Classification accuracy on generated data improved consistently over training epochs, providing a measurable indication of learning progress. Additional analysis examined class-wise performance and the effect of varying the input noise vector on output diversity.

Overall, the findings demonstrate that convolutional architectures are essential for high-quality image generation in GANs and that classifier-based evaluation provides a useful framework for benchmarking generative models.

1 Introduction

The goal of this research project is to use a Generative Adversarial Neural Network (GAN) [3] to create synthetic handwritten letters. Generative models have become an important area of machine learning due to their ability to learn complex data distributions and generate realistic samples that resemble real-world data. In particular, GANs have shown strong performance in generating high-quality images by training two neural networks in opposition: a generator that creates synthetic data and a discriminator that attempts to distinguish between real and generated samples. Below is a simplified illustration of a conditional GAN model

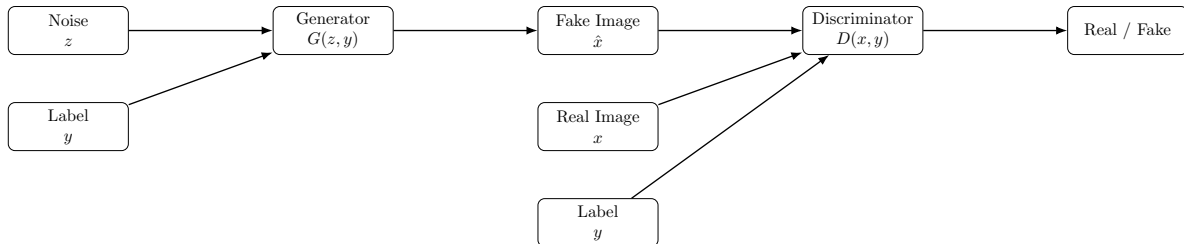


Figure 1: Conditional GAN (cGAN) architecture

In this project, the GAN is trained on the EMNIST Balanced dataset[2], which is an extension of the MNIST dataset containing both handwritten digits and letters. The dataset consists of 47 distinct classes: 10 classes representing the digits 0 to 9, and 37 classes representing uppercase and lowercase letters. Some letter classes were merged due to visual similarity, making classification more consistent and reducing ambiguity between certain character pairs.

The EMNIST Balanced dataset presents a more challenging task than standard digit recognition because of the increased number of classes and the variability in handwritten letter shapes. This makes it a suitable benchmark for evaluating the performance of generative models in learning complex character distributions.

Below is an image illustrating the distribution of samples across the different classes in the dataset:

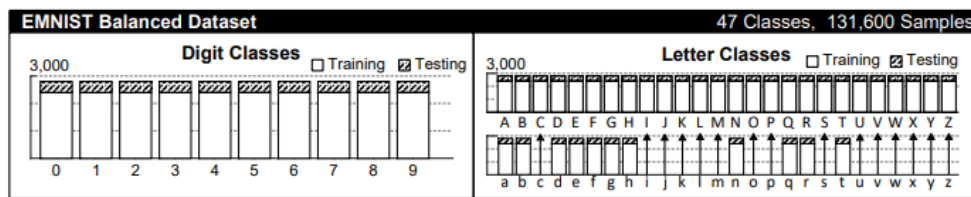


Figure 2: Class distribution of the EMNIST Balanced dataset

The objective of this work is not only to generate visually realistic handwritten characters but also to evaluate the quality of the generated samples using a trained classifier. By analyzing classification accuracy on synthetic data, we aim to measure how well the generator has learned the underlying structure of the dataset.

All computations were performed using the TensorFlow python library [1]. The all of the code, synthetic images and visualizations can be found on my GitHub account: <https://github.com/MicahJSherry/MATH418>

2 Traditional GAN

The first area of investigation was whether a GAN model without convolutional layers could be trained. We expect that this model will not perform well due to the highly complex and spatially structured nature of handwritten characters. Unlike convolutional architectures, fully connected layers do not preserve spatial relationships between pixels, making it difficult for the model to learn meaningful local patterns such as strokes and curves that are essential for realistic handwritten character generation.

2.1 Training Details

The conditional Generative Adversarial Network (cGAN) was trained using the EMNIST Letters dataset, which consists of handwritten characters spanning 26 alphabet classes. The dataset images were normalized to the range $[-1, 1]$ and reshaped to $28 \times 28 \times 1$ grayscale format.

The training process was conducted for a total of 200 epochs. The dataset contains approximately 62400 training images, which were shuffled and divided into batches of size 128 during training.

During each training step, the generator receives a random noise vector of dimension 100 concatenated with a class label, and attempts to generate realistic handwritten character images. The discriminator is trained simultaneously to distinguish between real and generated images conditioned on the same class labels.

The training was performed using the Adam optimizer with a learning rate of 2×10^{-4} . The adversarial loss was computed using binary cross-entropy for both the generator and discriminator networks.

To monitor progress, sample images for all 26 classes were generated every 5 epochs.

2.2 Results

The result of training a GAN model without convolutional layers provided unusable synthetic handwritten images. Figure 3 shows synthetic images generate after training for 200 epochs. Notice that all the images are highly similar to each other and do not resemble any letter.

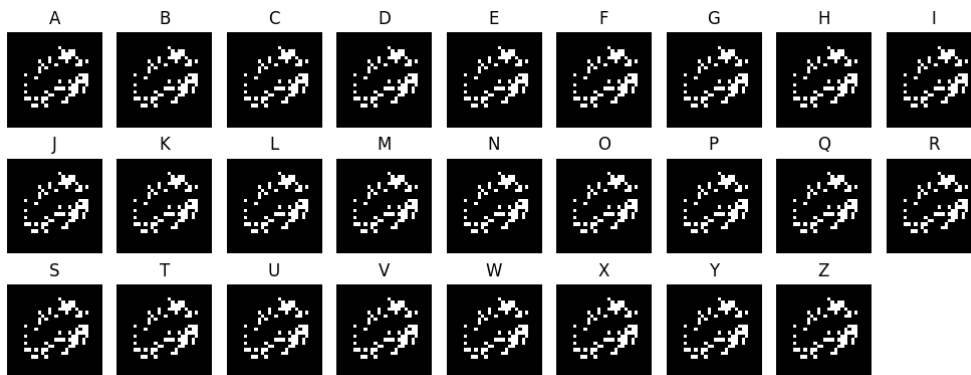


Figure 3: Synthetic Images Generated by GAN without Convolutional Layers

3 GAN With Convolutional Layers

Now that we know that GAN with traditional neural network layers does not perform well with this dataset. We now look at GAN with convolutional layers. Convolutional layers have proven to be a good choice for neural networks for image processing because allow the model to better detect local features and spacial relationships such as edges, corners, and curves which is extremely important to image classification.

3.1 Training Details

The training process was very similarity to the model without convolutional layers.

The model was trained using the EMNIST Balanced dataset, which consists of handwritten characters across 47 classes. The dataset images were normalized to the range $[-1, 1]$ and reshaped to $28 \times 28 \times 1$ grayscale format.

The training process was conducted for a total of 100 epochs. The dataset contained approximately 112,800 training images, which were shuffled and divided into batches of size **128** during training.

During each training step, the generator receives a random noise vector of dimension 100 concatenated with a learned class label embedding and produces synthetic handwritten character images. The discriminator

receives both real and generated images, along with their corresponding class labels, and learns to classify them as real or fake.

The generator architecture is based on convolutional transpose layers, which progressively upsample the input vector into a 28×28 image. The discriminator uses convolutional layers to extract hierarchical spatial features before performing binary classification.

The model was trained using the Adam optimizer with a learning rate of 2×10^{-4} and $\beta_1 = 0.5$. Binary cross-entropy was used as the adversarial loss function for both the generator and discriminator networks.

To monitor training progress, sample images for all 47 classes were generated every 5 epochs and saved for qualitative evaluation.

3.2 Results

As expected the model with convolutional layers performed very well (compared to traditional model). Even after 5 epochs the generated images(See Figure 5) are useable. And after 100 epochs the images (See Figure ??) appear very closely like handwritten characters.

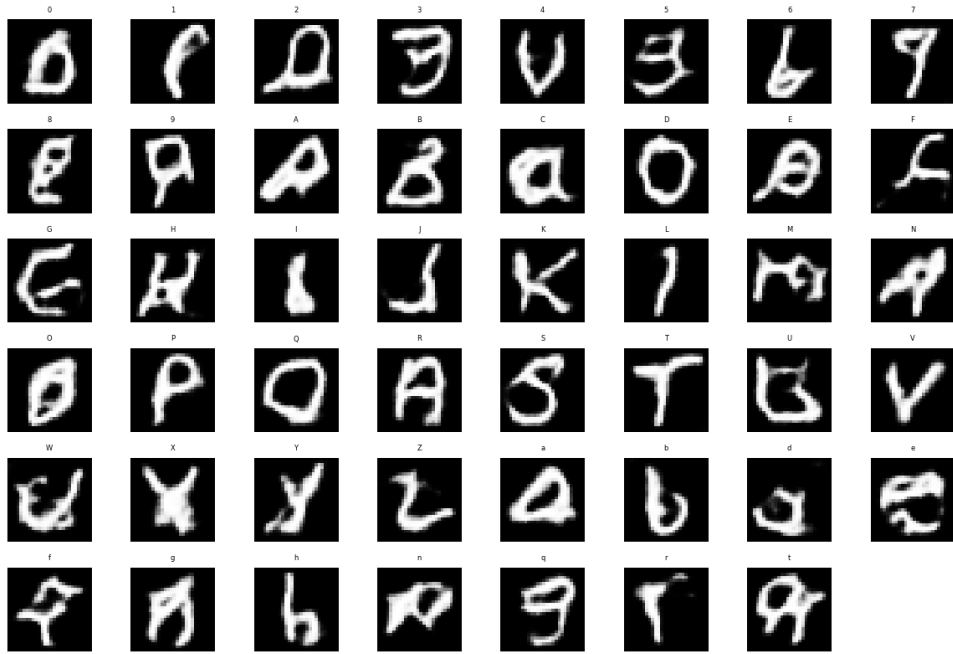


Figure 4: Synthetic images generated by GAN with Convolutional layers after 5 epochs

4 Benchmarking

Determining how well the model performs is difficult because of the subjectivity of Determining images looks better.

4.1 Classification Model

To get a Numerical Score of how well a model performed a classification model was trained on the EMNIST Balanced dataset. Then the generator was used to create a synthetic dataset. the Synthetic dataset was then scored on classification accuracy. Figure 6 shows classification accuracy of the synthetic dataset by epoch.

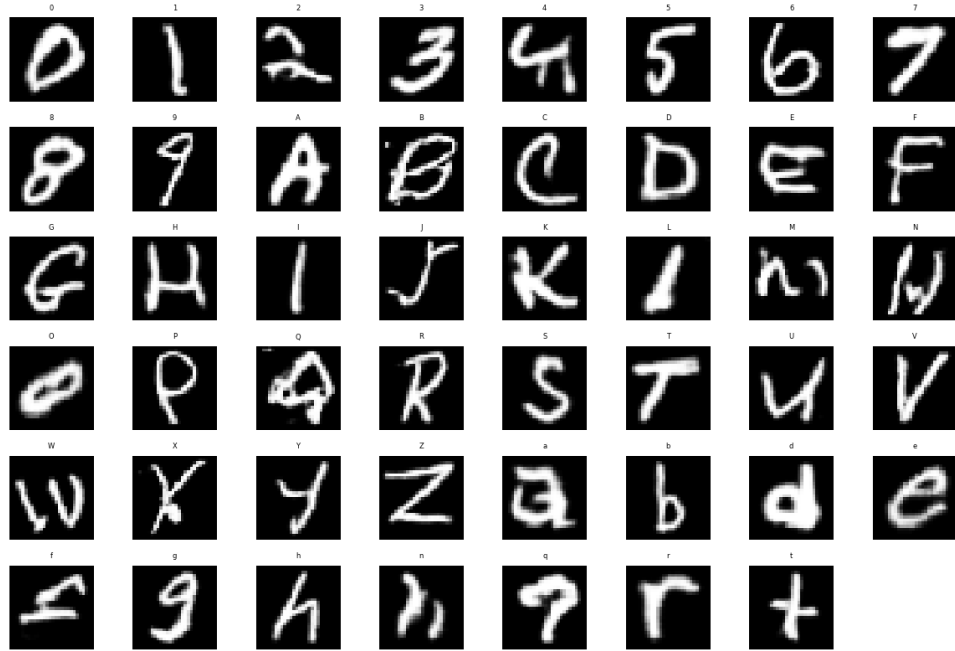


Figure 5: Synthetic images generated by GAN with Convolutional layers after 100 epochs

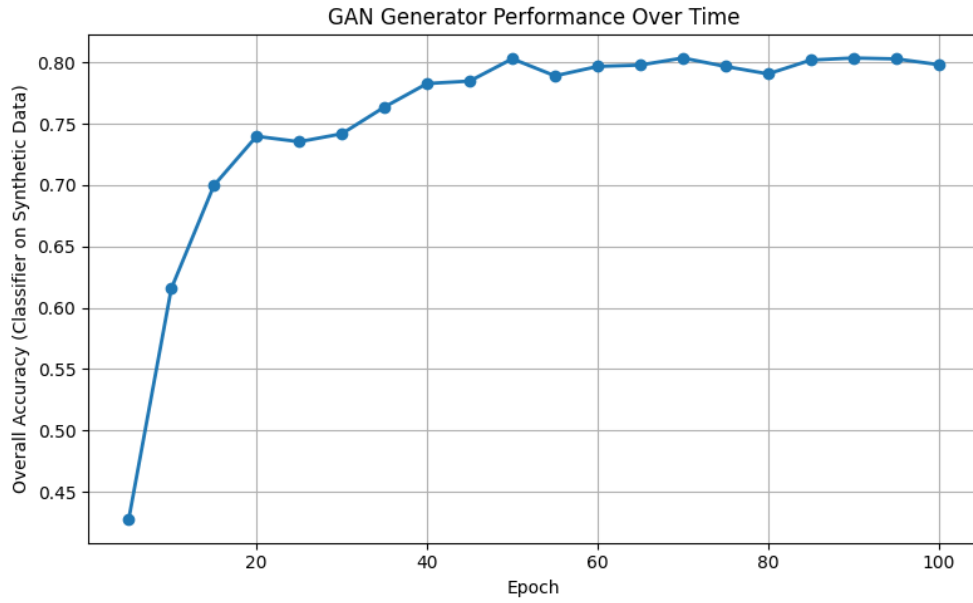


Figure 6: classification accuracy by epoch

We also want to look at the accuracy of each of the classes over the epochs. Which is shown in figure 7 as you can see the classes with the lowest accuracy tend to be similar to another class for example 0 and O or 9 and q and g.

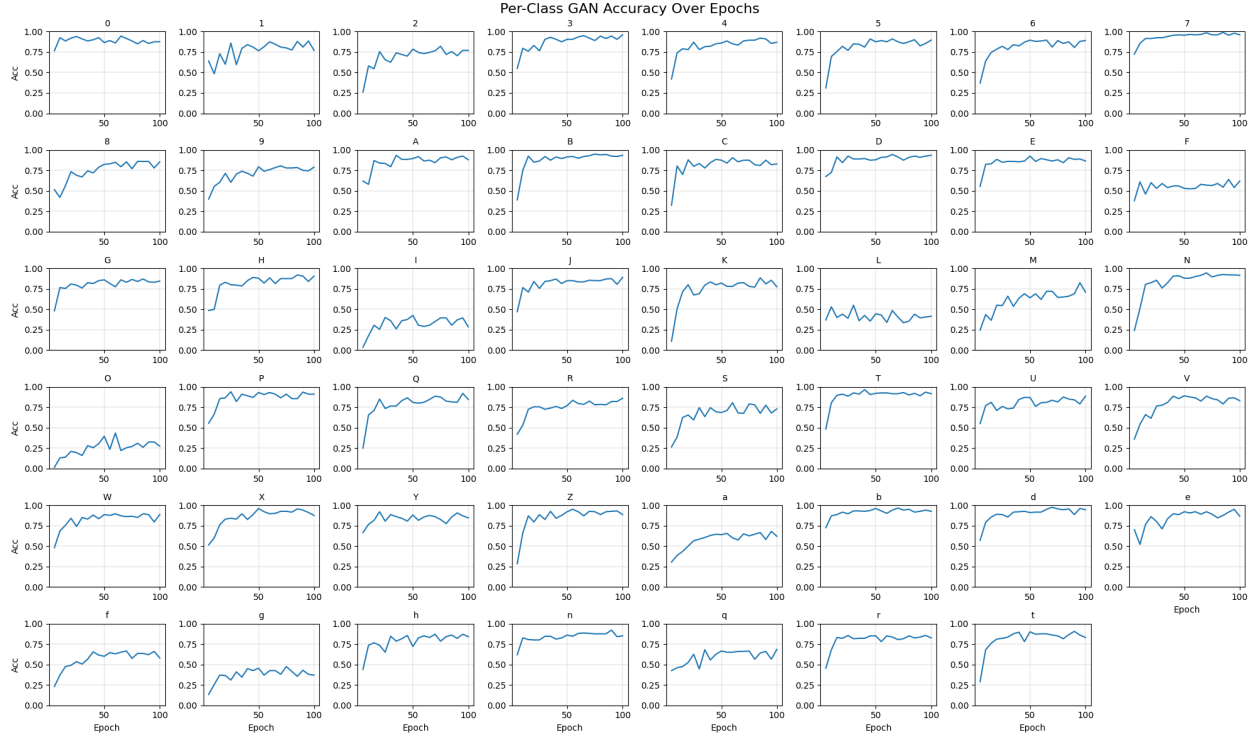


Figure 7: class classification accuracy by epoch

4.2 Progression Over Epochs

Next we look at the progression of the synthetic images over the epochs. The figures included are some of the more interesting progressions. To generate the images the same random noise vector was used to control for random variation.

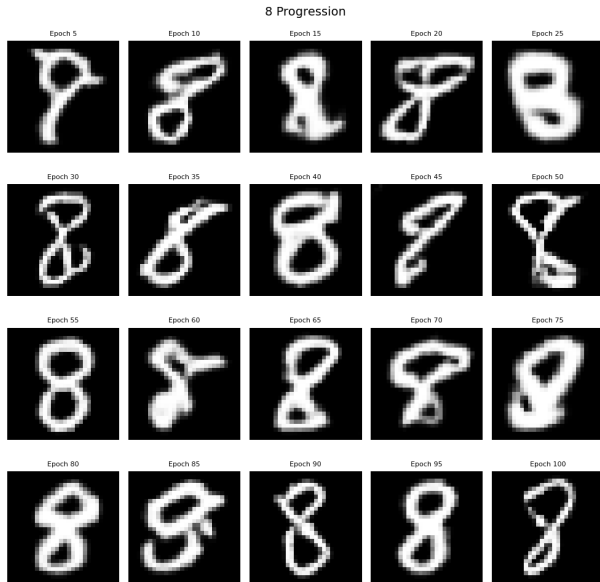


Figure 8: Progress synthetic generated '8's

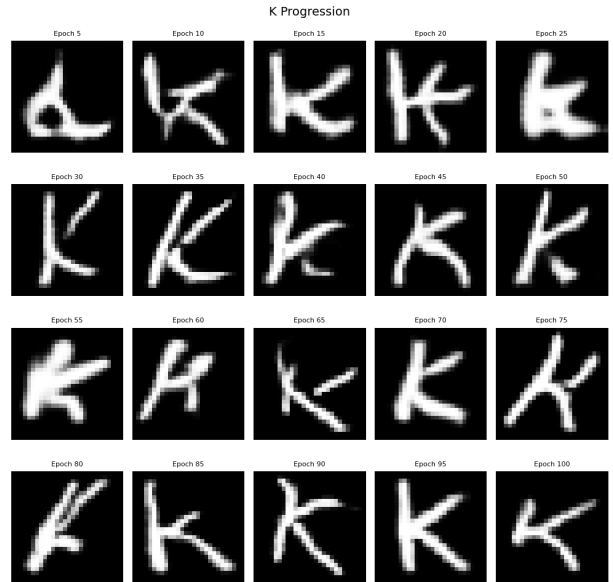


Figure 9: Progress synthetic generated 'K's

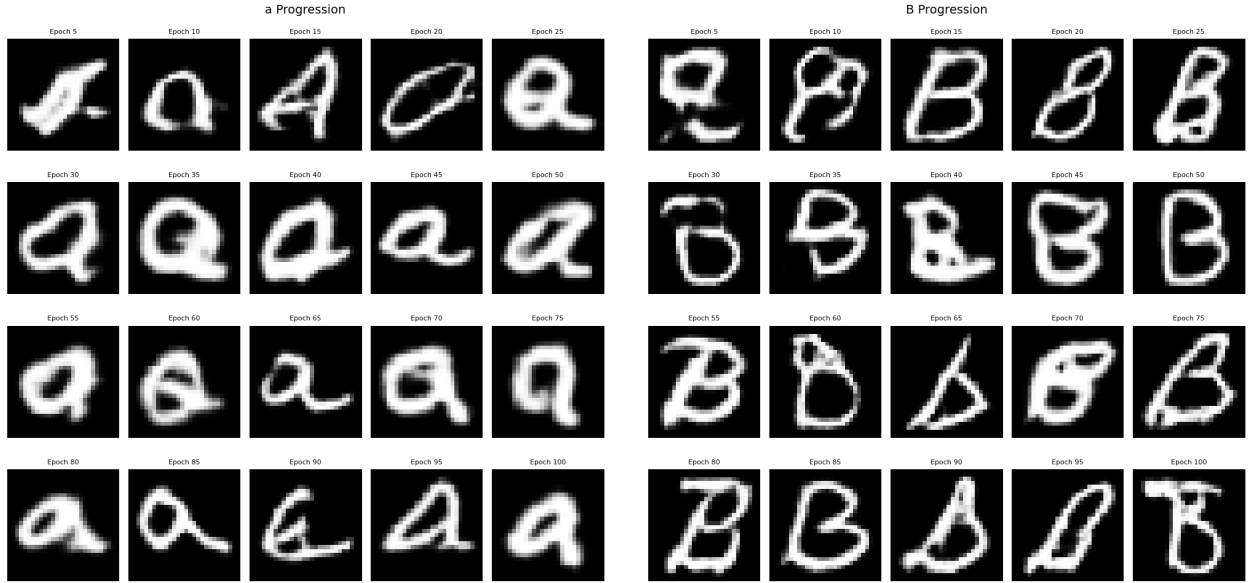


Figure 10: Progress synthetic generated 'a's

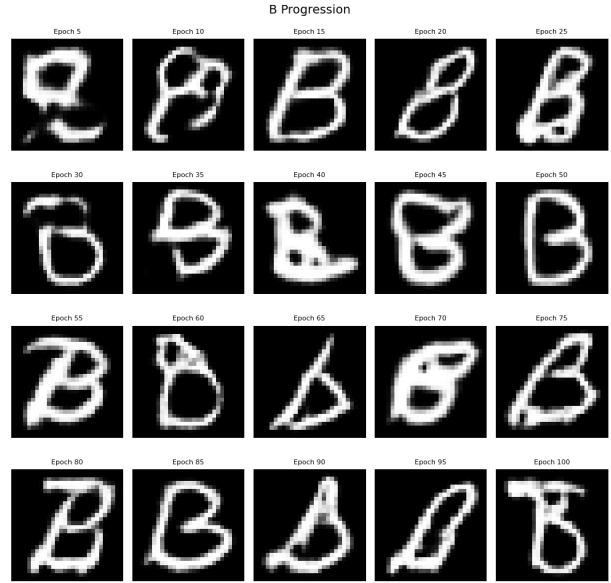


Figure 12: Progress synthetic generated 'B's

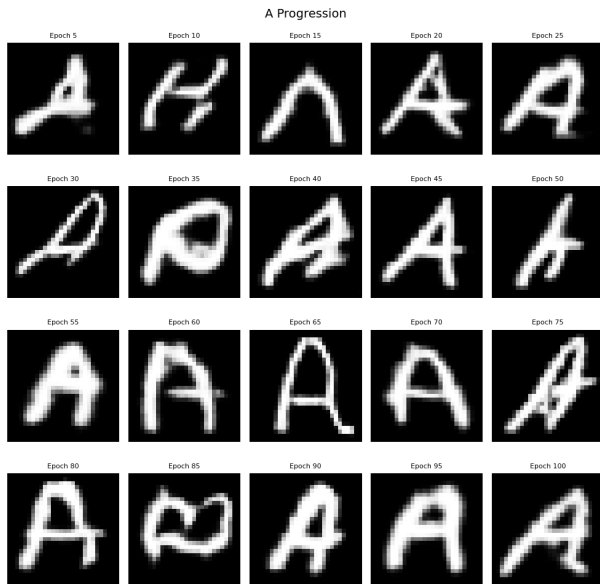


Figure 11: Progress synthetic generated 'A's

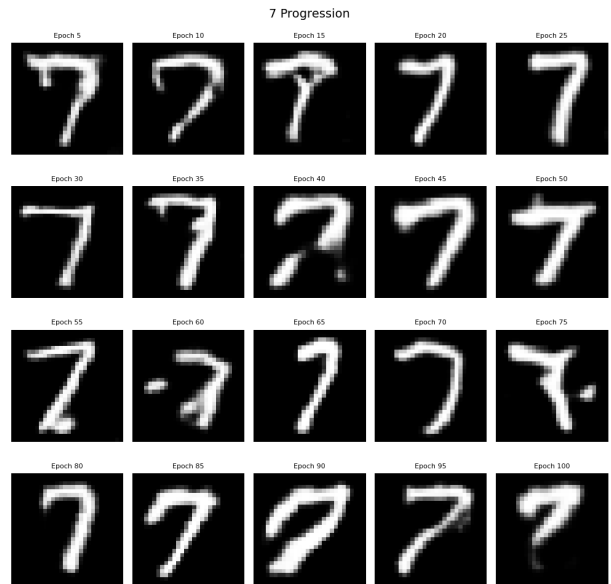


Figure 13: Progress synthetic generated '7's

4.3 Variation of Noise

Next to get a sense of how the generated images vary we generate synthetic from a specific epoch and vary the random noise vector to see how things change. The following figures are some of the more interesting ones.

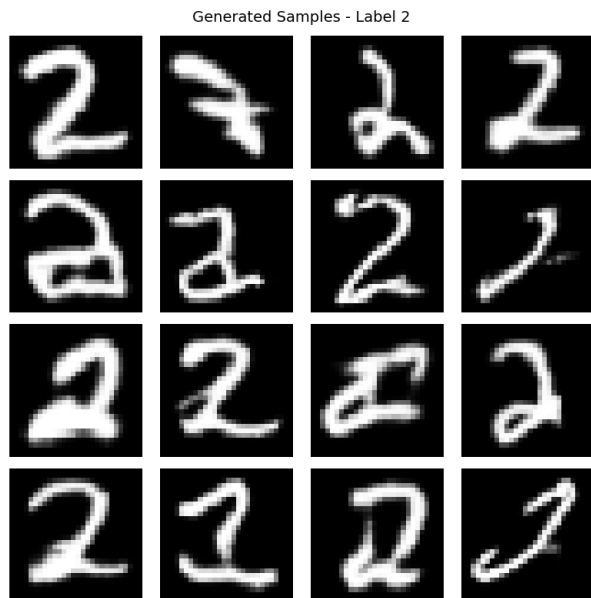


Figure 14: Varying Noise for '2'

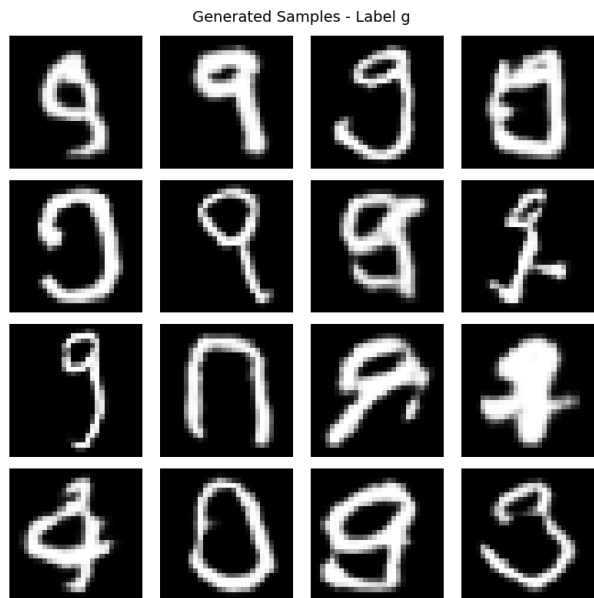


Figure 16: Varying Noise for 'g'

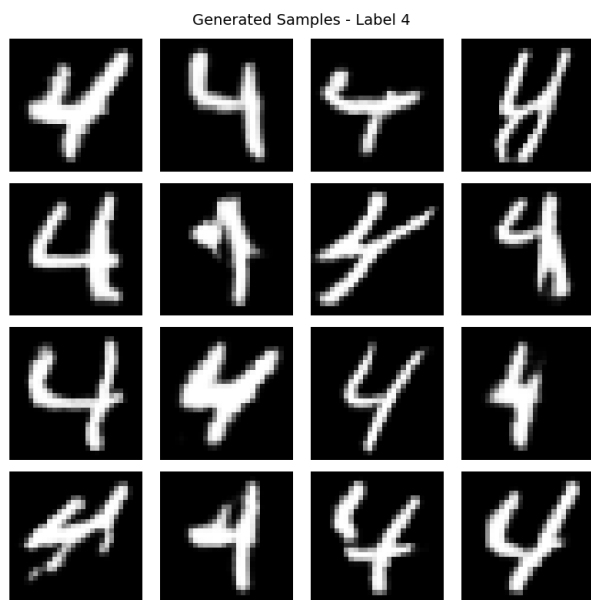


Figure 15: Varying Noise for '4'

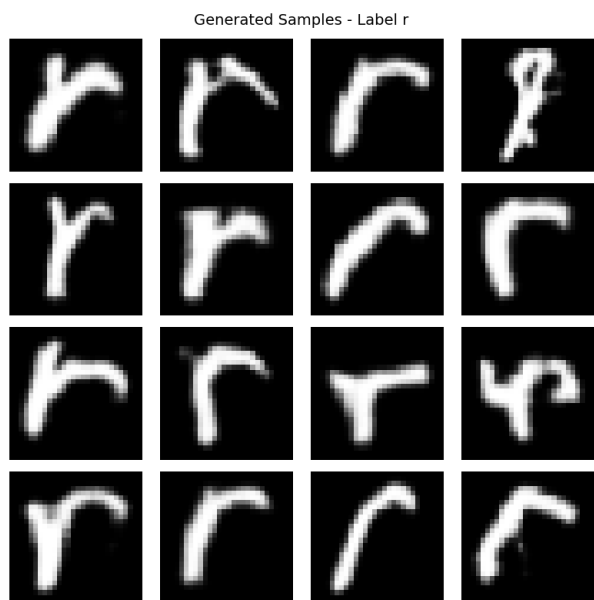


Figure 17: Varying Noise for 'r'

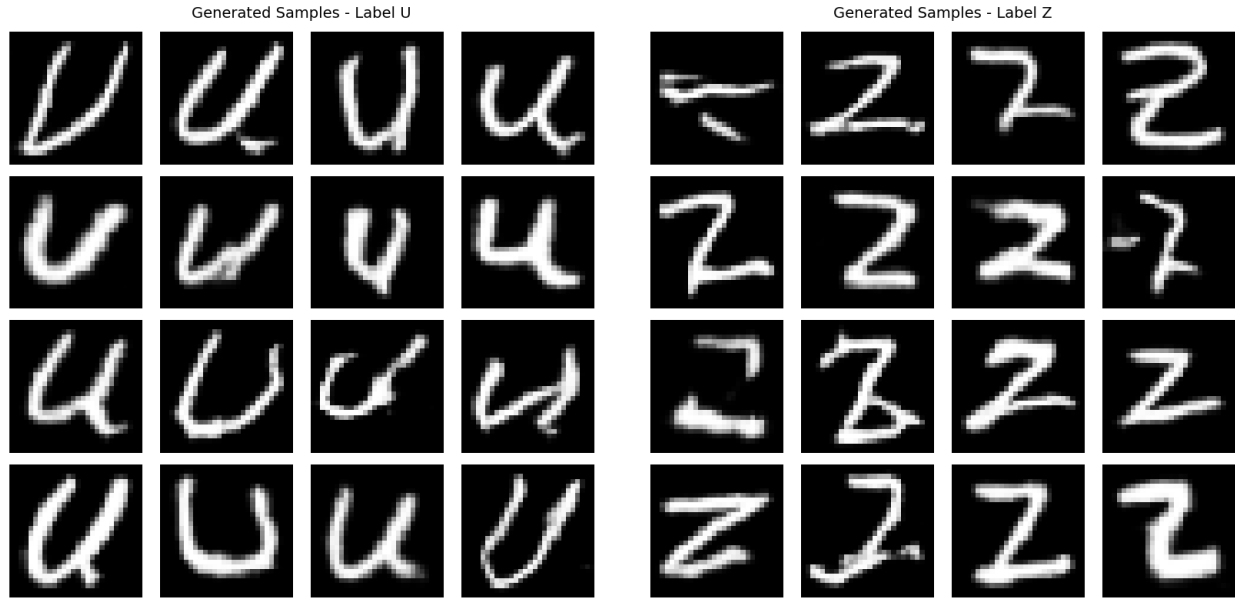


Figure 18: Varying Noise for 'U'

Figure 19: Varying Noise for 'Z'

5 Applications

There are many potential applications of this research. For this research project I implemented synthetic handwritten quote generations. For this application to work well the synthetic image quality needs to be improved as a proof of concept the following quotes(or strings of characters) were synthetically generated.

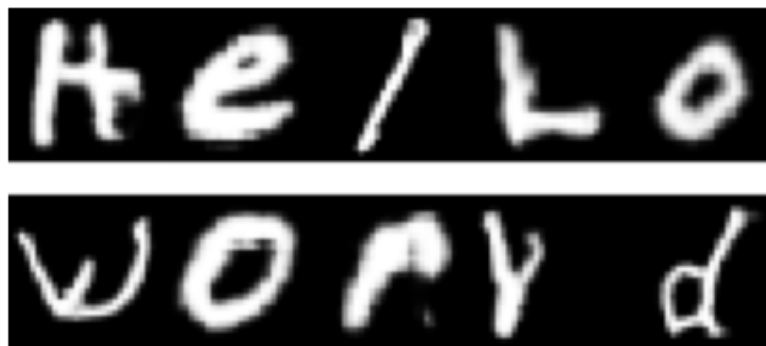


Figure 20: Generated handwritten text: "Hello world"

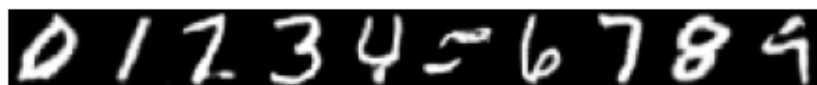


Figure 21: Generated handwritten digits sequence "0123456789"



Figure 22: Generated handwritten quote: "It is not enough to have a good mind the main thing is to use it well — René Descartes"

6 Further Study

There is many possible questions for further study some of which are:

1. Can the model be modified to emulate a single author's handwriting style?
2. Could a traditional GAN be effectively trained on the EMNIST dataset
3. What preprocessing methods could be applied to improve the results?
4. Could the noise vector be used to control for letter style?

5. Can the distinction between similar symbols be improved?
6. Comparison with conditional GAN individual GANs being trained on each symbol.

7 Conclusion

In this project, a conditional Generative Adversarial Network (cGAN) was used to generate synthetic handwritten characters from the EMNIST dataset. The results demonstrate that model architecture plays a critical role in performance, as the GAN without convolutional layers failed to capture meaningful structure in the data, while the convolutional GAN produced realistic and diverse handwritten characters.

Quantitative evaluation using a trained classifier provided a useful metric for assessing generator performance, showing clear improvement over training epochs. Additionally, qualitative analysis of generated samples revealed that the model was able to learn class-specific features, though some confusion remained between visually similar characters.

Overall, this work highlights the effectiveness of convolutional GANs for image generation tasks and demonstrates a practical approach for evaluating generative models using downstream classification performance. Future work could explore more advanced architectures, improved training stability techniques, or higher-resolution datasets to further enhance generation quality.

References

- [1] Martín Abadi, Ashish Agarwal, Paul Barham, Eugene Brevdo, Zhifeng Chen, Craig Citro, Greg S. Corrado, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Ian Goodfellow, Andrew Harp, Geoffrey Irving, Michael Isard, Yangqing Jia, Rafal Jozefowicz, Lukasz Kaiser, Manjunath Kudlur, Josh Levenberg, Dandelion Mané, Rajat Monga, Sherry Moore, Derek Murray, Chris Olah, Mike Schuster, Jonathon Shlens, Benoit Steiner, Ilya Sutskever, Kunal Talwar, Paul Tucker, Vincent Vanhoucke, Vijay Vasudevan, Fernanda Viégas, Oriol Vinyals, Pete Warden, Martin Wattenberg, Martin Wicke, Yuan Yu, and Xiaoqiang Zheng. TensorFlow: Large-scale machine learning on heterogeneous systems, 2015. Software available from tensorflow.org.
- [2] Gregory Cohen, Saeed Afshar, Jonathan Tapson, and André van Schaik. Emnist: an extension of mnist to handwritten letters, 2017.
- [3] Mehdi Mirza and Simon Osindero. Conditional generative adversarial nets, 2014.